Wait… DDPM Finally Makes Sense!

A mathematical derivation, one coefficient at a time

Chao-Hsu (Victor) Chen

Master of Science in Applied Computing

Department of Computer Science, University of Toronto

Abstract

Diffusion models generate images by turning Gaussian noise into data one small step at a time, yet the sampling equation of Ho et al. is often learned as a formula to memorize. This note derives it from scratch. A right triangle inscribed in a semicircle explains why the forward step uses square-root coefficients. Adding independent Gaussians (with dice as a warm-up for convolution) collapses the whole noising chain into one closed-form jump. Bayes' rule and completing the square then give the reverse posterior, and rewriting that posterior in terms of noise yields the DDPM sampling update and the simplified training loss. The goal is not a new method but understanding: every coefficient should be explainable well enough to re-derive without looking it up.

Keywords Diffusion models, DDPM, Gaussian convolution, Bayesian inference, generative models

1 Introduction

When I first read *Denoising Diffusion Probabilistic Models* [1], I could follow the training and sampling algorithms, but I could not see the steps that led to their equations. Why do the square roots appear? How does a chain of noisy images collapse into a single equation? And where does the reverse update actually come from?

These notes are my attempt to answer those questions from the ground up. We start with a semicircle, combine independent Gaussian noises, and use Bayes' rule to build the reverse step. Then we connect the result to the training objective. The aim is simple: to explain each coefficient, not just copy it.

The two algorithms we want to understand

A restatement of Algorithms 1 and 2 of Ho et al. [1]. The operations are the same; only the layout and notation are ours. Throughout, $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$.

Training (Algorithm 1). Repeat until convergence:

1. Draw $x_0 \sim q_{\text{data}}$, a step $t \sim \text{Uniform}\{1, \ldots, T\}$, and $\epsilon \sim \mathcal{N}(0, I)$.

2. Take a gradient step on

$$\nabla_\theta \left\| \epsilon - \epsilon_\theta\big(\underbrace{\sqrt{\bar\alpha_t}\, x_0 + \sqrt{1-\bar\alpha_t}\,\epsilon}_{x_t},\ t\big) \right\|^2 .$$

Sampling (Algorithm 2). Draw $x_T \sim \mathcal{N}(0, I)$. For $t = T, \dots, 1$:

1. Draw $z \sim \mathcal{N}(0, I)$ if $t > 1$; otherwise set $z = 0$.
2. Update

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\bar\alpha_t}}\, \epsilon_\theta(x_t, t) \right) + \sigma_t z.$$

Return x_0.

Our route

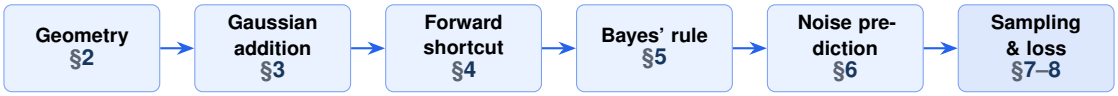


Figure 1. The derivation in one line. Each stage supplies exactly what the next one needs.

Scope and prerequisites

Scope. This tutorial follows my handwritten derivation and adds the detailed posterior algebra and the training objective where the notes left steps implicit. It is a route to the existing DDPM equations, not a new diffusion method.

What we need. Basic probability, Gaussian distributions, and algebra. Images are vectors in $\mathbb{R}^d$; the scalar arguments apply coordinate by coordinate because the noise is isotropic Gaussian.

2 Why the Square Roots?

Let's make the noise budget visible.

We begin with one forward step:

$$x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \epsilon_t, \qquad \epsilon_t \sim \mathcal{N}(0, I). \tag{1}$$

The new noise ϵ_t is independent of x_0 and of all earlier step noises. The schedule satisfies $0 < \beta_t < 1$; we usually increase β_t over time, but the derivation does not require it.

For reference, the scalar Gaussian density and scaling rule are

$$\mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\,\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2} \right), \quad \sigma > 0, \tag{2}$$

$$X \sim \mathcal{N}(\mu, \sigma^2) \implies cX \sim \mathcal{N}(c\mu, c^2\sigma^2) \quad (c \neq 0). \tag{3}$$

Why use $\sqrt{\beta_t}$ rather than β_t? Because scaling a random variable by c scales its variance by c^2. We want the new noise to have variance $\beta_t I$, so its amplitude must be $\sqrt{\beta_t}$.

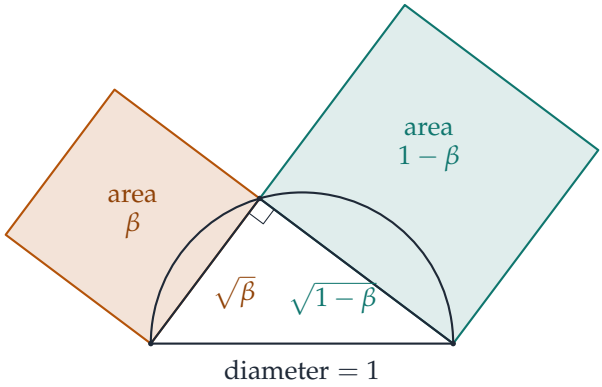


Figure 2. The semicircle and two squares from my notes (drawn with $\beta = 0.36$). A point on a semicircle makes a right angle with the diameter's endpoints (Thales' theorem), so the squares on the two legs have areas that add up to the square on the diameter, which is 1. This is a coefficient diagram, not a picture of an image or of a probability density.

The geometry gives us

$$(\sqrt{\beta_t})^2 + (\sqrt{1-\beta_t})^2 = 1.$$

For one coordinate, if x_{t-1} has variance v, independence gives

$$\mathrm{Var}(x_t) = (1-\beta_t)\, v + \beta_t.$$

So unit variance stays unit variance. If x_{t-1} has a different variance, we should not claim that its variance is automatically one. Likewise, the mean becomes $\sqrt{1-\beta_t}\,\mathbb{E}[x_{t-1}]$.

> **The picture's job**
> The side lengths explain *amplitudes*; the square areas explain *variance contributions*. The two coefficients do not add to one as amplitudes. Their *squares* add to one.

Set $\alpha_t = 1 - \beta_t$. Our one-step rule becomes

$$\begin{aligned} x_t &= \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \epsilon_t, \\ q(x_t \mid x_{t-1}) &= \mathcal{N}\big(\sqrt{\alpha_t}\, x_{t-1},\, \beta_t I\big). \end{aligned} \tag{4}$$

In words: shrink the previous image a little, then add fresh noise to refill the variance budget.

3 Adding Independent Gaussians

Two noise terms in, one noise term out.

After two steps, x_2 contains *two* independent noise terms. To merge them, we need one fact.

The rule

If $X \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $Y \sim \mathcal{N}(\mu_2, \sigma_2^2)$ are independent, then

$$X + Y \sim \mathcal{N}(\mu_1 + \mu_2,\ \sigma_1^2 + \sigma_2^2). \tag{5}$$

Means and variances add for *any* independent random variables. The special part is that the sum is again Gaussian. To see where such a statement comes from, we start with something we can count.

3.1 Warm-up: convolution with dice

Roll two independent, fair, six-sided dice X and Y, so $P(X = k) = \frac{1}{6}$ for $k = 1, \ldots, 6$. What is the probability that the sum is 2? Only $1 + 1$ works, and there are $6 \times 6 = 36$ equally likely outcomes, so $P(X + Y = 2) = \frac{1}{36}$. A sum of 3 can happen as $1 + 2$ or $2 + 1$, so $P(X + Y = 3) = \frac{2}{36}$ (Figure 3).

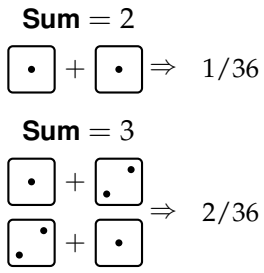


Figure 3. Counting combinations for dice sums: one way to obtain 2, two ways to obtain 3.

For any target sum z, we consider every pair $(k, z - k)$:

$$P(X + Y = z) = \sum_k P(X = k)\, P(Y = z - k). \tag{6}$$

This operation is called **discrete convolution**: it finds the distribution of a sum by combining all the ways of obtaining that sum. For two fair dice,

$$P(X + Y = z) = \frac{1}{36} \begin{cases} z - 1, & 2 \le z \le 7, \\ 13 - z, & 8 \le z \le 12. \end{cases}$$

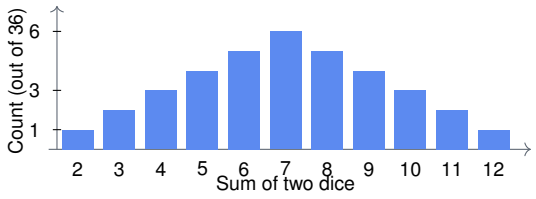


Figure 4. Distribution of the sum of two fair dice. Dividing each count by 36 gives its probability.

Sums near 7 have more combinations, which is why the distribution is triangular.

3.2 From dice to Gaussians

For continuous variables, we replace the sum by an integral of densities:

$$p_{X+Y}(z) = \int_{-\infty}^{\infty} p_X(x)\, p_Y(z-x)\, \mathrm{d}x. \tag{7}$$

The idea is exactly the dice idea: for each possible value x of the first variable, the second must equal $z - x$, and we add up all these contributions. If p_X and p_Y are both Gaussian, carrying out the integral (by completing the square in x) produces another Gaussian, which gives Eq. (5). We take this standard fact as given.

It lets us merge several independent Gaussian noise terms into one, which is precisely what the forward process needs.

4 The Forward Shortcut

Why we never have to simulate the whole chain.

Let x_0 be the original image. Starting from Eq. (4), we expand the first two steps:

$$x_1 = \sqrt{\alpha_1}\, x_0 + \sqrt{1-\alpha_1}\, \epsilon_1, \tag{8}$$

$$\begin{aligned} x_2 &= \sqrt{\alpha_2}\, x_1 + \sqrt{1-\alpha_2}\, \epsilon_2 \\ &= \sqrt{\alpha_1\alpha_2}\, x_0 + \underbrace{\sqrt{\alpha_2(1-\alpha_1)}\, \epsilon_1 + \sqrt{1-\alpha_2}\, \epsilon_2}_{\text{two independent Gaussian terms}}. \end{aligned} \tag{9}$$

By the scaling rule (3), the two noise terms have variances $\alpha_2(1-\alpha_1)$ and $1-\alpha_2$. By the addition rule (5), their sum has variance

$$\alpha_2(1-\alpha_1) + (1-\alpha_2) = 1 - \alpha_1\alpha_2.$$

Hence

$$x_2 \overset{d}{=} \sqrt{\alpha_1\alpha_2}\, x_0 + \sqrt{1-\alpha_1\alpha_2}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I), \tag{10}$$

where $\overset{d}{=}$ means equality *in distribution*, not equality of individual samples.

4.1 Any timestep

Define $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$ with $\bar{\alpha}_0 = 1$. Suppose that $x_{t-1} \overset{d}{=} \sqrt{\bar{\alpha}_{t-1}}\, x_0 + \sqrt{1-\bar{\alpha}_{t-1}}\, \bar{\epsilon}$. Then

$$x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \epsilon_t \overset{d}{=} \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{\alpha_t(1-\bar{\alpha}_{t-1})}\, \bar{\epsilon} + \sqrt{1-\alpha_t}\, \epsilon_t,$$

and the two noise variances add up to

$$\alpha_t(1-\bar{\alpha}_{t-1}) + (1-\alpha_t) = \alpha_t - \bar{\alpha}_t + 1 - \alpha_t = 1 - \bar{\alpha}_t.$$

By induction (the case $t = 2$ above is the first step),

$$x_t \overset{d}{=} \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I), \tag{11}$$

that is,

$$q(x_t \mid x_0) = \mathcal{N}\big(\sqrt{\bar{\alpha}_t}\, x_0,\ (1 - \bar{\alpha}_t) I\big). \tag{12}$$

Why this matters

It might seem that we must simulate $x_1, x_2, \ldots, x_{t-1}$ to obtain x_t. Eq. (11) says we can jump *directly* from x_0 to any t. This is what makes training efficient. It also shows the trend: the signal coefficient $\sqrt{\bar{\alpha}_t}$ shrinks while the noise coefficient $\sqrt{1 - \bar{\alpha}_t}$ grows, and their squares always add to one.

5 Reversing with Bayes' Rule

Adding noise was easy. Removing it requires conditioning.

Generation needs the reverse step: from x_t we want x_{t-1}, that is, $q(x_{t-1} \mid x_t)$. Unfortunately, this distribution depends on the unknown data distribution. So we first solve an easier problem: *what if we also knew x_0?*

5.1 Applying Bayes' rule

By Bayes' rule and the Markov property (x_t depends on x_{t-1} only),

$$q(x_{t-1} \mid x_t, x_0) = \frac{q(x_t \mid x_{t-1})\, q(x_{t-1} \mid x_0)}{q(x_t \mid x_0)}. \tag{13}$$

Every factor on the right is already known:

$$q(x_t \mid x_{t-1}) = \mathcal{N}\big(\sqrt{\alpha_t}\, x_{t-1},\ \beta_t I\big), \tag{14}$$

$$q(x_{t-1} \mid x_0) = \mathcal{N}\big(\sqrt{\bar{\alpha}_{t-1}}\, x_0,\ (1 - \bar{\alpha}_{t-1}) I\big), \tag{15}$$

$$q(x_t \mid x_0) = \mathcal{N}\big(\sqrt{\bar{\alpha}_t}\, x_0,\ (1 - \bar{\alpha}_t) I\big). \tag{16}$$

Since all three are Gaussian, we can find the posterior by completing the square.

5.2 Completing the square

Let $y = x_{t-1}$. The denominator in (13) does not depend on y, so

$$q(y \mid x_t, x_0) \;\propto\; \exp\left[-\frac{\|x_t - \sqrt{\alpha_t}\, y\|^2}{2\beta_t} - \frac{\|y - \sqrt{\bar{\alpha}_{t-1}}\, x_0\|^2}{2(1-\bar{\alpha}_{t-1})} \right]. \tag{17}$$

Expanding, every term that involves y fits the quadratic form

$$-\tfrac{1}{2}\big(A\, y^\top y - 2\, b^\top y + C\big), \tag{18}$$

where C does not depend on y, and

$$A = \frac{\alpha_t}{\beta_t} + \frac{1}{1-\bar{\alpha}_{t-1}}, \qquad b = \frac{\sqrt{\alpha_t}}{\beta_t}\, x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_{t-1}}\, x_0. \tag{19}$$

A Gaussian $\propto \exp[-\frac{1}{2}(A\, y^\top y - 2 b^\top y)]$ has covariance $A^{-1} I$ and mean $A^{-1} b$, so

$$\tilde{\beta}_t = A^{-1}, \qquad \tilde{\mu}_t = A^{-1} b. \tag{20}$$

5.3 Posterior variance

Combining the fractions in A^{-1},

$$\tilde{\beta}_t = \left(\frac{\alpha_t}{\beta_t} + \frac{1}{1-\bar{\alpha}_{t-1}} \right)^{-1} = \frac{\beta_t (1-\bar{\alpha}_{t-1})}{\alpha_t (1-\bar{\alpha}_{t-1}) + \beta_t}.$$

The denominator collapses, because $\alpha_t(1-\bar{\alpha}_{t-1}) + \beta_t = \alpha_t - \bar{\alpha}_t + 1 - \alpha_t = 1 - \bar{\alpha}_t$. Therefore

$$\boxed{\tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\, \beta_t.} \tag{21}$$

5.4 Posterior mean

Multiplying b by $\tilde{\beta}_t$ term by term gives

$$\tilde{\mu}_t = \frac{\sqrt{\alpha_t}\,(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\, x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\, x_0. \tag{22}$$

Putting everything together:

$$\boxed{q(x_{t-1} \mid x_t, x_0) = \mathcal{N}\big(\tilde{\mu}_t,\ \tilde{\beta}_t I\big).} \tag{23}$$

If we know both x_t and x_0, we can describe x_{t-1} exactly. But there is a catch: *during generation we do not know x_0.*

6 From Predicting Images to Predicting Noise

The unknown x_0 can be traded for the unknown noise.

Return to the forward shortcut $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon$ and solve for x_0:

$$x_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\, \epsilon}{\sqrt{\bar{\alpha}_t}}. \tag{24}$$

So instead of asking a network to predict x_0 directly, we can ask it to predict the noise ϵ. Let $\epsilon_\theta(x_t, t)$ denote this prediction.

Substituting (24) into the posterior mean (22), and using $\sqrt{\bar{\alpha}_{t-1}}/\sqrt{\bar{\alpha}_t} = 1/\sqrt{\alpha_t}$:

$$\tilde{\mu}_t = \left[\frac{\sqrt{\alpha_t}\,(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} + \frac{\beta_t}{\sqrt{\alpha_t}\,(1-\bar{\alpha}_t)}\right] x_t - \frac{\beta_t}{\sqrt{\alpha_t}\sqrt{1-\bar{\alpha}_t}}\, \epsilon. \tag{25}$$

The bracket simplifies nicely:

$$\frac{\alpha_t(1-\bar{\alpha}_{t-1}) + \beta_t}{\sqrt{\alpha_t}\,(1-\bar{\alpha}_t)} = \frac{1-\bar{\alpha}_t}{\sqrt{\alpha_t}\,(1-\bar{\alpha}_t)} = \frac{1}{\sqrt{\alpha_t}}.$$

Therefore

$$\boxed{\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\, \epsilon\right).} \tag{26}$$

The posterior mean is now written with noise instead of the original image. Replacing the unknown ϵ by the network's prediction gives the model mean

$$\boxed{\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\, \epsilon_\theta(x_t, t)\right).} \tag{27}$$

Reading the formula

(1) The network estimates the noise inside x_t. (2) We subtract a scaled copy of that estimate, with scale $\beta_t/\sqrt{1-\bar{\alpha}_t}$. (3) We divide by $\sqrt{\alpha_t}$, undoing the shrinkage the forward step applied to the signal. Each piece has a job; none is arbitrary.

7 The DDPM Sampling Equation

The learned reverse transition is modeled as

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\big(\mu_\theta(x_t, t),\, \sigma_t^2 I\big). \tag{28}$$

To sample from a Gaussian with mean μ and standard deviation σ, we use $x = \mu + \sigma z$ with $z \sim \mathcal{N}(0, I)$ (again the scaling rule). Substituting (27),

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z, \qquad z \sim \mathcal{N}(0, I). \tag{29}$$

This is the DDPM reverse sampling equation, matching Algorithm 2. We started from Gaussian properties, simplified the forward process, used Bayes' rule for the posterior, and replaced the unknown noise with a network prediction. We no longer need to memorize it, because we know where every term comes from.

7.1 What about the variance?

How do we choose σ_t^2? In DDPM it is a fixed schedule rather than learned. Two choices discussed in the original paper are

$$\sigma_t^2 = \beta_t \qquad \text{or} \qquad \sigma_t^2 = \tilde{\beta}_t.$$

Ho et al. report similar results for both. These are *modeling choices*: the exact conditional posterior in Eq. (23) has variance $\tilde{\beta}_t I$, but that does not mean the unconditional reverse distribution has the same variance. Improved DDPM [2] also investigates learning the reverse-process variance. At the final step ($t = 1$) we set $z = 0$, so the last update returns the mean.

8 Training the Noise Predictor

We generate the noise ourselves, so we already know the answer.

To train $\epsilon_\theta(x_t, t)$, we need a target for the noise. Since we add the noise ourselves, we know it exactly. We sample

$$x_0 \sim q_{\text{data}}, \qquad t \sim \text{Uniform}\{1, \dots, T\}, \qquad \epsilon \sim \mathcal{N}(0, I),$$

build $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon$ directly from Eq. (11), and feed (x_t, t) to the network. The simplified objective is

$$L_{\text{simple}} = \mathbb{E}_{x_0, t, \epsilon} \left[\left\| \epsilon - \epsilon_\theta\left(\sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon,\ t \right) \right\|^2 \right]. \tag{30}$$

This is the commonly used simplified DDPM loss, motivated by the variational bound in the original paper (it drops the per-timestep weighting of that bound). Because of the forward shortcut, training never runs the whole chain.

9 Putting Everything Together

Cheat sheet: three equations and where each comes from

Forward step squares of the coefficients add to one (§2)

$$x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{\beta_t}\, \epsilon_t$$

Forward shortcut Gaussian addition applied t times (§3–4)

$$x_t \overset{d}{=} \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon$$

Reverse sampling Bayes' posterior with x_0 rewritten via noise (§5–7)

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\, \epsilon_\theta(x_t, t)\right) + \sigma_t z$$

Training

1. Sample a clean image x_0.
2. Choose a random timestep t.
3. Sample Gaussian noise ϵ.
4. Construct x_t directly.
5. Predict $\epsilon_\theta(x_t, t)$.
6. Minimize the noise-prediction loss.

Generation

1. Start from $x_T \sim \mathcal{N}(0, I)$.
2. For $t = T, \ldots, 1$, apply Eq. (29) with fresh z (and $z = 0$ at $t = 1$).
3. The final x_0 is the generated image.

10 Conclusion

At first, the DDPM sampling equation can look complicated. Derived step by step, it follows from a few ideas. The semicircle explains the square-root coefficients. Convolution explains how independent noises combine. Gaussian addition collapses the forward chain into one jump. Bayes' rule gives the reverse posterior, and a network's estimate of the noise stands in for the unknown image.

The key takeaway is that DDPM does not simply subtract noise from an image: it learns a *probabilistic* reverse process, one small Gaussian step at a time. Our next step is to implement these equations in PyTorch and visualize how images evolve during forward and reverse diffusion.

» Github Code Link

References

[1] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:2006.11239.

[2] A. Q. Nichol and P. Dhariwal, "Improved Denoising Diffusion Probabilistic Models," *International Conference on Machine Learning (ICML)*, 2021. arXiv:2102.09672.